# ECEN 749 Lab 1 - Report

Yashas Bharadwaj
UIN: 635009332; Section: 601

## Introduction

This lab is an introduction to programming the Zynq-7000 series Xinilix FPGA board using Vivado. It included implementing a switch as a basic introductory program, a 4-bit counter (up and down) and a jackpot game. This lab also serves as an introductory class to interfacing different peripherals like switches, push buttons and LEDs.

## Procedure

1. There were two files given to us, a verilog file and a .xdc file which is a constraint file. The first exercise was to implement a 4-bit up and down counter. The up counter was to be done with a push button (BTN0) and the down counter with the other push button (BTN1). The counter module has four inputs (CLOCK, RESET, BTN0, BTN1) and one output (4 LEDS). The LEDS were used to visually see the implementation of the four bit counter.
2. For the second exercise, again there were two files, a verilog file and a .xdc file which is a constraint file. This exercise was to implement a jackpot game. The module 3 inputs CLOCK, RESET, and 4 SWITCHES. LEDs keep blinking in a sequence, if the DIP switch corresponding to that LED is turned ON at the same time as the light on that LED, then you win the jackpot and all four LEDs glow. Then you reset the game using one of the push buttons declared in the program and restart the game.

## Results

1. For the first exercise the clock divider logic should be used to decrease the frequency of the clock from 125MHz to 1Hz for our eyes to perceive. It was done by using the counter which would be set to 0 when its value would be exactly at 50% duty cycle of the original frequency. And at this time the clock would toggle. Thus a full cycle would have two toggles and output frequency would be 125MHz/125MHz = 1Hz.
   The main counter logic consists of an always block with the divided clock and a reset button in the sensitivity list. The reset button was also defined in the .xdc file using the push buttons K19. If reset was true the counter would be 0. If BTN0 is pushed and BTN1 isn't pushed, the counter would be incremented. If BTN1 is pushed and BTN0 is not, then the counter decrements. All these cases are visible through the LEDs present on the FPGA.
2. For the jackpot game, the same clock divider logic was used. The main LED machine consists of an always block with the same sensitivity list as the counter. There are four register variables led_state, prev, LEDS, and game_done which is 0 by default but when the jackpot is hit, the game_done will be 1. If the SWITCHES[state] && ~prev[state] && led_state is 1, then it is a jackpot. All the LEDs would light up and the game is done. Now to reset the game, a push button was assigned for RESET.

## Conclusion

The exercise questions of counter and jackpot were implemented using verilog and their respective constraint files. Familiarity with the Zybo board and its peripherals was done along with programming with verilog HDL.

## Questions

a) Pin K18 BTN0 - up counter
   Pin P16 BTN1 - down counter
   Pin K19 BTN2 - RESET
   Pin Y16 BTN3 - not used
   These are the four push buttons present in the board. By default the push buttons are pulled down so they provide a logic low as the output.

b) The purpose of edge detection circuits is to observe when the logic state changes from either low to high or high to low. In counter logic, it was used to see when the button was pressed and in the jackpot game, it was used to see when the switch was ON to trigger the jackpot as and when the conditions were met.

## Appendix

### Counter

/home/grads/y/yashas1729/ECEN749_Yashas/Lab1/Lab1.srcs/sources_1/new/counter.v

```verilog
`timescale 1ns / 1ps

module counter(
    output [3:0] LEDS,
    input BTN0,
    input BTN1,
    input CLOCK,
    input RESET
);

    reg [3:0] counter;
    reg [26:0] clk_div_count;  // enough to count up to 125_000_000
    reg slow_clk;

    // Clock Divider: Divide CLOCK to generate a slower clock (slow_clk)
    always @(posedge CLOCK or posedge RESET) begin
        if (RESET) begin
            clk_div_count <= 0;
            slow_clk <= 0;
        end else begin
            if (clk_div_count == (125000000/2 - 1)) begin
                clk_div_count <= 0;
                slow_clk <= ~slow_clk;
            end else begin
                clk_div_count <= clk_div_count + 1;
            end
        end
    end

    // Counter Logic: Triggered on slow clock
    always @(posedge slow_clk or posedge RESET) begin
        if (RESET) begin
            counter <= 0;
        end else begin
            if (BTN0 && !BTN1) begin
                counter <= counter + 1;
            end else if (!BTN0 && BTN1) begin
                counter <= counter - 1;
            end
            // If both or none are pressed, retain value (do nothing)
        end
    end

    assign LEDS = counter;

endmodule
```

MobaTextEditor

File Edit Search View Format Syntax Special Tools

counter.xdc

```xdc
#clock
set_property PACKAGE_PIN K17 [get_ports CLOCK]
set_property IOSTANDARD LVCMOS33 [get_ports CLOCK]

#reset
set_property PACKAGE_PIN K19 [get_ports RESET]
set_property IOSTANDARD LVCMOS33 [get_ports RESET]

#Push buttons
set_property PACKAGE_PIN K18 [get_ports BTN0]
set_property IOSTANDARD LVCMOS33 [get_ports BTN0]
set_property PACKAGE_PIN P16 [get_ports BTN1]
set_property IOSTANDARD LVCMOS33 [get_ports BTN1]

# LEDs
set_property PACKAGE_PIN M14 [get_ports {LEDS[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[0]}]
set_property PACKAGE_PIN M15 [get_ports {LEDS[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[1]}]
set_property PACKAGE_PIN G14 [get_ports {LEDS[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[2]}]
set_property PACKAGE_PIN D18 [get_ports {LEDS[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[3]}]
```

## Jackpot

/home/grads/y/yashas1729/ECEN749_Yashas/Lab1/Lab1.srcs/sources_1/new/jackpot.v

```verilog
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/14/2025 03:42:08 PM
// Design Name:
// Module Name: jackpot
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module jackpot{output reg[3:0] LEDS, input [3:0] SWITCHES, input CLOCK, input RESET};

reg [3:0] led_state;
reg game_done;
reg [31:0] counter;
reg [3:0] prev;

reg [26:0] clk_div_count;  // enough to count up to 125_000_000
reg slow_clk;

    // Clock Divider: Divide CLOCK to generate a slower clock (slow_clk)
    always @{posedge CLOCK or posedge RESET} begin
        if {RESET} begin
            clk_div_count <= 0;
            slow_clk <= 0;
        end else begin
            if {clk_div_count == {125000000/2 - 1}} begin
                clk_div_count <= 0;
                slow_clk <= ~slow_clk;
            end else begin
                clk_div_count <= clk_div_count + 1;
            end
        end
    end

//LED state machine
always @{posedge slow_clk or posedge RESET} begin
    if {RESET} begin
```

```
            end
        end
    end

//LED state machine
always @(posedge slow_clk or posedge RESET) begin
    if (RESET) begin
        led_state <= 4'b0001;
        game_done <= 0;
        LEDS <= 4'b0000;
        prev <= 4'b0000;
        end
    else begin
        if (!game_done) begin
            if (((SWITCHES[0] && ~prev[0]) && led_state == 4'b0001) ||
            ((SWITCHES[1] && ~prev[1]) && led_state == 4'b0010) ||
            ((SWITCHES[2] && ~prev[2]) && led_state == 4'b0100) ||
            ((SWITCHES[3] && ~prev[3]) && led_state == 4'b1000)) begin
                LEDS <= 4'b1111;
                game_done <= 1;
        end
        else begin
            case (led_state)
                4'b0001: led_state <= 4'b0010;
                4'b0010: led_state <= 4'b0100;
                4'b0100: led_state <= 4'b1000;
                4'b1000: led_state <= 4'b0001;
                default: led_state <= 4'b0001;
            endcase
            LEDS <= led_state;
        end
        end
        prev <= SWITCHES;
        end
end


endmodule

```

jackpot.xdc

```
#clock
set_property PACKAGE_PIN K17 [get_ports CLOCK]
set_property IOSTANDARD LVCMOS33 [get_ports CLOCK]

#reset
set_property PACKAGE_PIN K19 [get_ports RESET]
set_property IOSTANDARD LVCMOS33 [get_ports RESET]


# switches
set_property PACKAGE_PIN G15 [get_ports {SWITCHES[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[0]}]
set_property PACKAGE_PIN P15 [get_ports {SWITCHES[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[1]}]
set_property PACKAGE_PIN W13 [get_ports {SWITCHES[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[2]}]
set_property PACKAGE_PIN T16 [get_ports {SWITCHES[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[3]}]

# LEDs
set_property PACKAGE_PIN M14 [get_ports {LEDS[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[0]}]
set_property PACKAGE_PIN M15 [get_ports {LEDS[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[1]}]
set_property PACKAGE_PIN G14 [get_ports {LEDS[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[2]}]
set_property PACKAGE_PIN D18 [get_ports {LEDS[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[3]}]
```